Life Planner

Prepared for CA Munazza Parande

A Python desktop planner for professional responsibilities, personal tasks and a daughter's school activities.

The problem it solves

Work deadlines and family commitments compete for attention. This planner brings them into one place, highlights tomorrow's schedule, reminds the user of unfinished tasks and keeps completed work separate.

What has been built

• Three task categories: Work, Daughter / School, and Home / Personal.

• Today, Tomorrow, Next 7 days, All active and Completed views.

• Task entry with due time, reminder lead, notes and repeat frequency.

• Desktop reminder popups, sound, snooze and persistent local storage.

• Completion history, automatic next recurring occurrence and CSV export.

Project positioning

This is an **AI-assisted software development project**: a coding assistant helped produce the Python application. The delivered app itself uses deterministic scheduling rules and does not contain a machine-learning model or an AI scheduling engine. Describe it accurately in any demonstration or course submission.

Package contents

life_planner.py - application source code.
Run_Life_Planner.bat - Windows launcher.
START_HERE.txt - installation and operating instructions.

The reusable prompt

The prompt below reconstructs the requirements of the delivered application. It is a reusable specification, rather than a verbatim record of an earlier internal prompt. Copy the complete text into a coding assistant.

Prompt begins

Act as a senior Python developer. Build a Windows desktop application called **Life Planner** for CA Munazza Parande to manage work commitments, home responsibilities and her daughter's school activities. Use Python 3, Tkinter/ttk and SQLite from the standard library. Avoid third-party dependencies, API keys and subscriptions. Give the interface a clean lavender and indigo theme with readable text.

• Provide the categories Work, Daughter / School, and Home / Personal. Each task must store a title, category, due date and time, recurrence, reminder lead in minutes, notes, completion status and next reminder time.

• Provide Today, Tomorrow, Next 7 days, All active and Completed views, plus category filtering. Open on Tomorrow. Show counts for today, tomorrow and overdue tasks. Display overdue tasks in red; retain them in All active.

• Allow adding, editing and deleting tasks. Validate that the title is present, the date/time uses YYYY-MM-DD HH:MM and reminder lead is an integer from 0 to 10080. Ask before deleting.

• Support None, Daily, Weekdays (Monday-Friday), Weekly and Monthly recurrence. Completing a task must archive it in Completed and create one next future occurrence in the same database transaction. Skip missed repeats. Preserve scheduled time. Clamp monthly dates to the last valid day and document that subsequent months use the clamped date.

• Check reminders every 20 seconds while the app runs. At the lead time show a desktop popup and play the system bell. Repeat unfinished reminders every 30 minutes. Include a 10-minute snooze that does not change the due date. Resume overdue reminders on reopening.

• Store data under LOCALAPPDATA/MunazzaLifePlanner/planner.sqlite3 on Windows. Provide CSV export of all tasks. Keep data offline; do not connect email, school or calendar accounts. Use the computer local clock.

• Warn on exit that closing stops reminders. Explain that the app must remain running and the computer awake; minimizing is allowed. Do not claim mobile or closed-app notifications.

• Deliver life_planner.py, a Run_Life_Planner.bat launcher that tries py -3 then python, and START_HERE.txt in a ZIP. Explain Python/Tkinter installation if missing. Check syntax and verify recurrence and completion behavior; clearly state if the Windows GUI was not tested.

Prompt ends

How it was created

1. Define the workflow

Translate the need into a task lifecycle: add a task, show it in the appropriate date view, remind the user, and move it to Completed when finished. Create a new future task only when a recurring task is completed.

2. Choose a lightweight Python stack

Use Tkinter and ttk for windows, forms, buttons and tables; sqlite3 for persistence; datetime and calendar for scheduling; csv for exports; pathlib and os for local data paths. These modules require no pip packages.

3. Create the database

Create a tasks table on first launch. Store id, title, category, due, recurrence, lead, notes, done and notified. Store dates as YYYY-MM-DD HH:MM so they can be sorted consistently. The notified field records the next eligible reminder time.

4. Build the desktop interface

Create the main Planner class. Add the date and category selectors, summary counts, task table and action buttons. Build a separate task editor for entry and changes. Refresh the view after every change.

5. Implement recurring dates

Write next_due() to calculate a daily, weekday, weekly or monthly successor. In complete(), mark the original row done and insert the next future occurrence in one transaction. Skip past occurrences to prevent a backlog of duplicate tasks.

6. Add reminder processing

Use Tkinter root.after() to run check() periodically without a blocking sleep. Compare the current time with the due time minus the lead, or the stored snooze/repeat time. Show a popup, ring the system bell and persist the next alert time.

7. Add export and the launcher

Export the database rows to UTF-8 CSV through a save dialog. Create a Windows batch file that changes to its own folder, finds Python and launches the application. Include a plain-language setup guide.

8. Check and package

Compile the Python source for syntax errors. Verify weekday/weekend handling, monthly month-end handling and weekly dates. Exercise the actual completion method against an in-memory database to check that repeated completion does not create duplicate successors. Package the three deliverable files into a ZIP.

Validation boundary: Syntax and recurrence/completion checks passed during development. The desktop GUI and Windows launcher were not executed on a Windows computer in the development environment.

Using your planner

Run it on Windows

• Download Life_Planner.zip and use Extract All. Keep all extracted files together.

• Install Python 3 with Tcl/Tk support if it is not already installed. Enable Add Python to PATH during installation.

• Double-click Run_Life_Planner.bat. If launching manually, run: python life_planner.py

• Add real tasks using your computer's local date and time. Keep one app instance open, and minimize it when working elsewhere.

A short demonstration

Add a Work task such as "Client follow-up" for tomorrow. Add a Daughter / School task such as "Pack activity materials" and include a note about what to carry. Show the Tomorrow view and category filter. Then create a Daily task, mark it complete and show the original in Completed and the next future occurrence in All active.

To demonstrate a reminder, add a task due a few minutes from now with reminder lead set to 0. Leave the app running until its due time, then show the popup and sound. Choose the task in the main window and use Snooze to defer its next alert.

Practical limits and backups

Reminders require the app to run and the computer to remain awake. There are no phone, WhatsApp, email or closed-app alerts. A repeat appears only after the current task is completed. Monthly repeats can shift after a short month. Times follow the computer's clock; there is no separate time-zone conversion.

CSV export provides a readable record, but there is no CSV import. For a full backup, close the app and copy planner.sqlite3 from the MunazzaLifePlanner folder under LOCALAPPDATA to a safe location. Restoring that file is a manual operation. Task data is local and is not encrypted by the application.

Source and technical references

The project-specific behavior in this guide is based on the delivered life_planner.py and START_HERE.txt files. The following official references explain the underlying standard-library components:

Tkinter user interface: https://docs.python.org/3/library/tkinter.html

SQLite database access: https://docs.python.org/3/library/sqlite3.html

Date and time handling: https://docs.python.org/3/library/datetime.html

CSV export: https://docs.python.org/3/library/csv.html

Python for Windows: https://www.python.org/downloads/windows/